SYMBIOSIS INSTITUTE OF TECHNOLOGY, HYDERABAD
Constituent of Symbiosis International (Deemed University), Pune
(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)
Re-Accredited by NAAC with 'A++' Grade
॥वसुधैव कुटुम्बकम्॥

PROJECT REPORT

ON

"PRICETIME: MULTI-CRYPTOCURRENCY PAPER TRADING PLATFORM"

Subject: Data Structures - Lab

Submitted By:

Name	Branch	PRN
N. Lakshman Reddy	CSE	25070721038
Aditya Sampara	CSE	25070721057

SYMBIOSIS INSTITUTE OF TECHNOLOGY, HYDERABAD

INDEX

Acknowledgement **3**

1. Abstract and Introduction **4**

2. Methodology **6**

3. Data Structures Implemented **10**

3.1 Binary Search Tree (BST) 10
3.2 Doubly Linked List (DLL) 11
3.3 Singly Linked List (SLL) 13
3.4 Circular Linked List (CLL) 14
3.5 Queue 15
3.6 Stack 16

4. Algorithms **17**

4.1 Searching Algorithms 17
4.2 Sorting Algorithms 18
4.3 Use of These Algorithms in the Reports Section 19
4.4 FIFO Accounting for Realised Profit and Loss 19

5. Price-Time Matching Engine and System Workflow **20**

6. Technologies, Applications and Future Scope **23**

7. Practical Implementation **25**

7.1 Sign-in and the Public Structures Desk 25
7.2 Trade Desk 27
7.3 Portfolio 29
7.4 Reports 30
7.5 CSV Order Replay 30
7.6 Automated Tests 31
7.7 Visuals Not Included 31

8. Conclusion **32**

9. References **33**

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to **Symbiosis Institute of Technology, Hyderabad**, for giving us the opportunity to carry out this project as part of the **Data Structures - Lab** course. The laboratory environment and the course material gave us the foundation on which PriceTime is built.

We thank all the faculty members who taught and guided us during the Data Structures course. The concepts of linked lists, stacks, queues, trees, searching and sorting that we studied in class are the same concepts that run inside the PriceTime matching engine, and this project would not have been possible without that learning.

We also thank the open-source community. PriceTime is built with React, TypeScript, Node.js, PostgreSQL and other freely available tools, and their documentation helped us at every stage of development.

Finally, we are grateful to our families and friends for their constant support and encouragement while we designed, built, tested and documented this project.

1. ABSTRACT AND INTRODUCTION

1.1 Abstract

PriceTime is a multi-cryptocurrency **paper trading platform**: a web application in which a user trades seven cryptocurrencies (BTC, ETH, DOGE, XRP, ADA, TRX and AVAX) using **demo funds only**. No real money, deposit, withdrawal or real cryptocurrency transaction is involved. Every new account receives a demo balance of $500,000 and starts with no crypto holdings.

The central idea of the project is that the trading screen is only the visible part of the system. Behind it runs a **price-time priority matching engine** written from scratch in TypeScript, in which each classic data structure has a real job: a **binary search tree** keeps the price levels sorted, a **doubly linked list** holds the first-in-first-out queue of orders at each price, a **singly linked list** records the execution tape, a **circular linked list** keeps a rolling window of the last 20 trade prices, and a **stack** keeps an audit log of submit and cancel actions. Binary search finds orders by id, linear search filters the reports, and five sorting algorithms (bubble, selection, insertion, merge and quick sort) order the report tables. Realised profit and loss is calculated with **FIFO lot accounting**.

The application is a React and TypeScript web client, a Node.js and Hono API that runs as a Neon Function, Neon Auth for sign-in, and a Neon PostgreSQL database.

1.2 Introduction

Online exchanges let many buyers and sellers trade the same asset at the same time. To do this fairly, an exchange keeps an **order book**: a record of every buy and sell order that is still waiting to be filled. When a new order arrives, the exchange must decide, very quickly, which waiting order it should be matched with. This decision is a data-structure problem. The book has to be searched, kept in order, updated at both ends, and scanned in sequence, and each of these operations suits a different structure.

1.3 Problem Statement

Trading with real money is risky for a beginner, and many practice simulators hide how orders are actually matched. At the same time, textbook data structures are usually taught with small examples that do not show why a particular structure was chosen. Beginners who want to learn trading often have nowhere safe to practise, and students often see data structures only in isolated exercises. The problem addressed by this project is therefore twofold: (1) to provide a safe paper-trading environment with a visible and correct matching process, and (2) to show, in a working system, where each data structure and algorithm from the Data Structures course is the right tool.

1.4 Importance of Data Structures in Trading Systems

A matching engine must repeatedly answer four questions: what is the best price on each side, who is first in line at that price, where is a particular order, and what happened most recently. Choosing the wrong structure makes one of these questions slow. A sorted array makes insertion costly, an unsorted list makes finding the best price costly, and a singly linked queue makes cancelling an order from the middle costly. The structures used in PriceTime are chosen so that each question has an efficient answer, as Table 1 summarises.

Table 1: *Questions an order book must answer and the structure PriceTime uses*

Question	Structure used	Reason
Which price is best right now?	Binary search tree (one per side)	Maximum or minimum node is found by walking one edge of the tree.
Who is first at this price?	Doubly linked list used as a FIFO queue	The head is the oldest order; it is read in O(1).
What was executed, in order?	Singly linked list (tape)	Each trade is appended at the tail in O(1).
What is the recent average price?	Circular linked list (20 slots)	A fixed window that overwrites the oldest value.
What did the engine just do?	Stack	Last action is on top; push and peek are O(1).
Where is order number k?	Binary search on a sorted array	Order ids only increase, so the array stays sorted.

1.5 How PriceTime Simulates Trading

A signed-in user chooses a cryptocurrency, a side (buy or sell), an order type (limit or market), a quantity and, for a limit order, a price. The browser sends this to the API, which first checks the account (enough demo cash for a buy, enough units for a sell), then passes the order to the matching engine. The engine matches it against the resting orders of other participants according to price-time priority and produces zero or more trades. The result is saved to the database in a single transaction, and the screen refreshes with the new order book, executions, balances and profit and loss. A seeded account named *paper-market-maker* keeps standing orders in every book so that a new user has someone to trade with.

Two points must be stated clearly. First, all money in PriceTime is demo cash, and all fills are simulated inside PriceTime's own order book. Second, the TradingView chart and the Binance reference prices are only displayed to make the screen look like a trading terminal; orders are never sent to Binance or to any real exchange, and the chart liquidity is never matched against.

1.6 Role of the Matching Engine

The matching engine (`apps/api/src/engine/orderBook.ts`) is the core of the project. It owns one order book per cryptocurrency, accepts submit and cancel requests, enforces price-time priority, creates trade records, and exposes a snapshot of the book (best bid, best ask, spread, top price levels, recent trades and moving average). It knows nothing about users, money or the database, so the same engine runs on the server and in the browser on the public Structures desk.

1.7 Objectives

- To build a working multi-cryptocurrency paper trading platform that uses demo funds only.
- To implement a price-time priority matching engine from first principles, without a trading library.
- To use a binary search tree, doubly, singly and circular linked lists, a queue and a stack in places where they are the natural choice, and to explain why.
- To apply linear and binary search and five sorting algorithms to real data in the Reports and Structures screens.
- To calculate realised and unrealised profit and loss correctly using FIFO lot accounting.
- To store all records in PostgreSQL inside atomic transactions, protect the API with signed tokens, test the engine with an automated suite and deploy the application on the web.

2. METHODOLOGY

2.1 Requirement Analysis

The requirements were identified from the aim of the project and divided into functional and non-functional groups.

Table 2: *Functional and non-functional requirements*

Type	Requirement
Functional	Register and sign in with email and password; receive demo cash on first use.
Functional	Choose one of seven cryptocurrencies and view its chart, best bid, best ask and spread.
Functional	Place limit and market orders on the buy or sell side; cancel an open order.
Functional	Match orders by price priority and then time priority, with partial fills.
Functional	Refuse an order that would trade against the user's own resting order.
Functional	Show cash, holdings, cost basis, realised and unrealised profit and loss.
Functional	Provide reports with searching and sorting, a CSV replay of orders, and a public Structures desk that visualises the data structures.
Non-functional	Correctness: an order is either fully recorded or not recorded at all (transaction).
Non-functional	Security: every account-related request needs a valid signed token; input is validated.
Non-functional	Clarity: no real funds; the interface states that fills are simulated.
Non-functional	Maintainability: typed code, a Drizzle schema with migrations, and automated tests.

2.2 System Architecture

PriceTime has three tiers. The **web client** is a static React application served from GitHub Pages. The **API** is a Hono application written for Node.js and run as a Neon Function. The **database** is Neon PostgreSQL. Sign-in is handled by Neon Auth, which issues a signed JSON Web Token (JWT) to the browser. The client sends this token with each API request, and the API verifies it against the public keys (JWKS) published by Neon Auth before doing anything else. Figure 1 shows the arrangement.

2.3 Frontend Design

The client (`apps/web`) is built with React 18, TypeScript and Vite. After sign-in it has four pages, reached from a sidebar: **Trade** (ticker strip, key figures, chart, order ticket, order book, recent executions and open orders), **Portfolio** (cash, positions, cost basis and profit and loss), **Reports** (executions with linear search, binary search and sorting) and **Replay** (CSV order replay and benchmark). A fifth page, **Structures**, is public and runs the matching engine in the browser on a workload that the visitor edits. A small module (`lib/api.ts`) attaches the bearer token to every request, and `lib/auth.ts` wraps the Neon Auth client. The TradingView Advanced Chart widget is loaded for the symbol `BINANCE:XXXUSDT` purely for display.

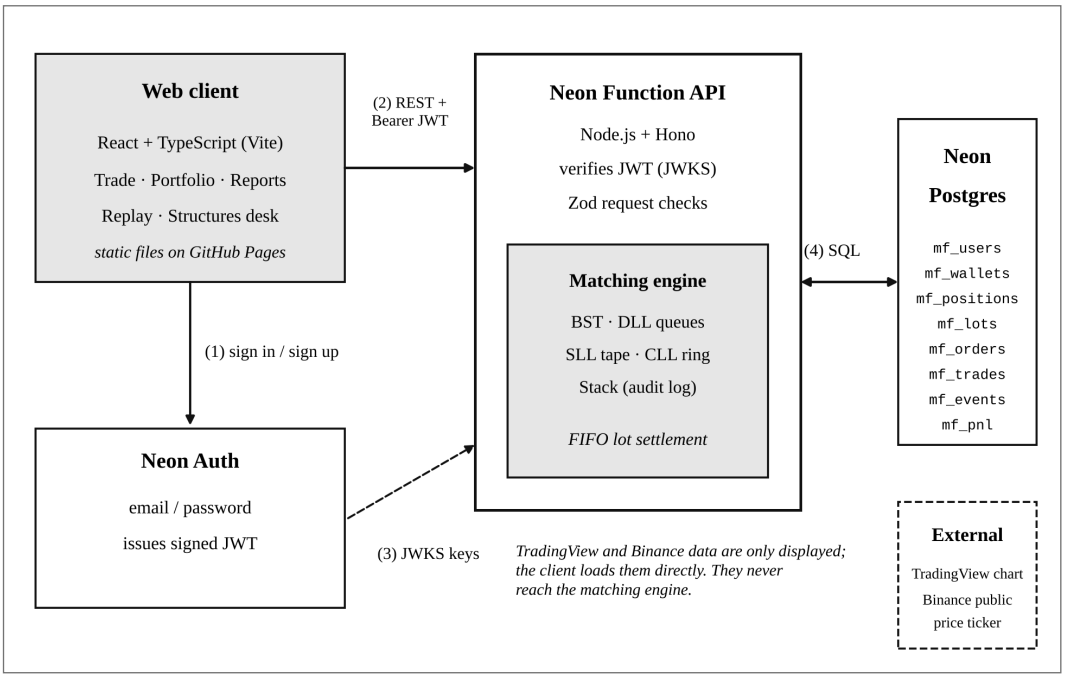


Figure 1: *System architecture of PriceTime. The matching engine runs inside the API; TradingView and Binance data are displayed by the browser only. The database holds nine tables (the eight main ones are listed, plus mf_replay_runs for replay benchmarks).*

2.4 Backend Design

The API (`apps/api/src/function.ts`) is a single Hono application. A middleware layer reads the `Authorization: Bearer` header, verifies the token with `jose` against the remote JWKS and the expected issuer, and stores the user's id, email and name for the request. Request bodies are validated with Zod: the symbol must be one of the seven supported symbols, the side must be BUY or SELL, the type LIMIT or MARKET, the quantity a positive integer up to 1,000,000, and a limit order must have a price. Table 3 lists the endpoints.

Table 3: *REST endpoints of the PriceTime API*

Method and path	Purpose
GET /health	Database health check (no sign-in required).
GET /api/symbols	The seven symbols with names and reference prices.
GET /api/account	Wallet, positions, realised and unrealised profit and loss.
GET /api/books and /api/books/:symbol	Order-book snapshot: best bid and ask, spread, top levels, recent trades, moving average.
GET /api/trades	Executions in which the user took part.
GET /api/orders	The user's orders and their status.
POST /api/orders	Submit a limit or market order.
DELETE /api/orders/:symbol/:id	Cancel an open or partially filled order and release the reserved funds.
POST /api/replay	Replay a CSV of orders through the BST/DLL engine and a plain-array baseline.

2.5 Data Structure Implementation

All structures are written by hand in TypeScript, without a library. The engine folder contains `orderBook.ts` (the price tree, the price levels with their doubly linked queues, the order book, the execution tape), `dataStructures.ts` (the circular linked list, the stack and a standalone doubly linked queue) and `algorithms.ts` (searching and sorting). Chapter 3 describes each structure in detail.

2.6 Matching Engine

The engine receives an order and repeatedly compares it with the best opposite price. If the prices cross, it trades with the oldest order at that price, reduces both quantities, records a trade, and removes finished orders and empty price levels. Any unfilled part of a limit order is added to the book; any unfilled part of a market order is released. Chapter 5 gives the rules and a worked example.

2.7 Database Integration

The schema is defined in Drizzle ORM (`apps/api/src/db/schema.ts`) and applied with Drizzle migrations. It has nine tables: users, wallets, positions, portfolio lots, orders, trades, order events, replay runs and realised profit and loss (all prefixed `mf_`). A distinctive design choice is that the order book is **event-sourced**. The engine's state is not stored directly. Instead every submit and cancel is stored in `mf_order_events` with an increasing sequence number, and for each request the API rebuilds the in-memory book by replaying these events in order. Writes are serialised with a PostgreSQL advisory transaction lock, and all effects of one order (wallet, positions, lots, orders, trades, events and realised profit and loss) are committed in a single transaction, so a failed request leaves no partial changes. Prices are stored as whole integer cents to avoid rounding errors.

Note on naming: the code was first written for a currency in paise, so price fields are named `pricePaise`. After the project moved to US dollars, these fields hold **cents**. This report always shows prices in dollars.

2.8 Authentication

Neon Auth (based on BetterAuth) provides email and password sign-up and sign-in. On success the browser holds a session and obtains a JWT. The API never sees the password; it only verifies the signed token. If the token is missing or invalid the API answers with HTTP 401 and the client asks the user to sign in again.

2.9 Trading Workflow

Figure 2 shows the path of one order through the system, from the user to the final reports.

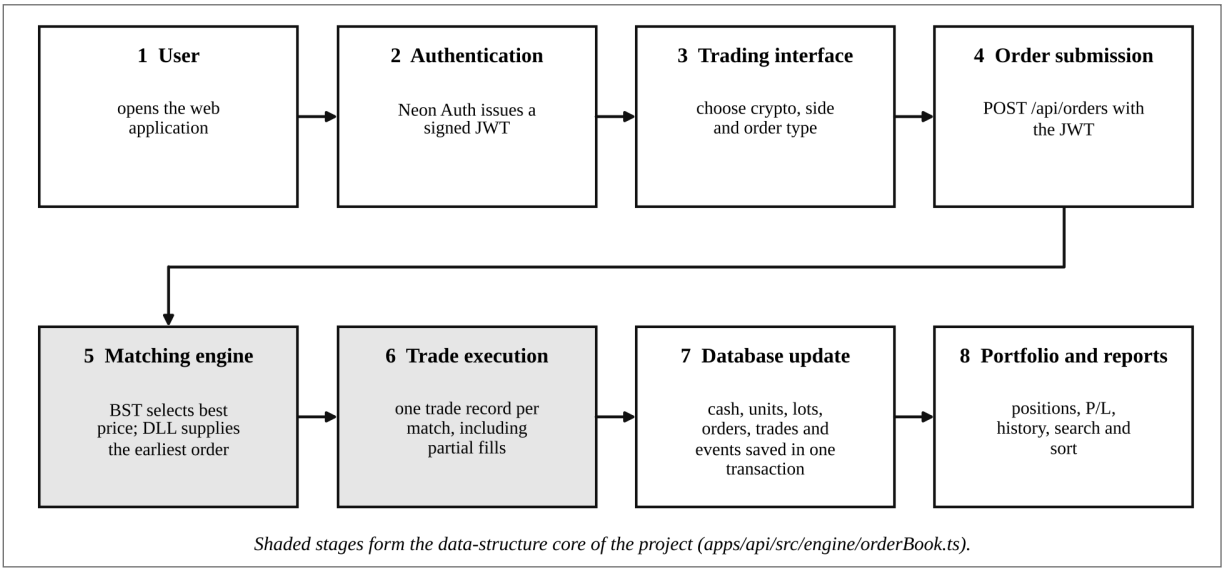


Figure 2: *Workflow of an order: User, Authentication, Trading Interface, Order Submission, Matching Engine, Trade Execution, Database Update, Portfolio and Reports.*

Step-by-step user interaction

- The user opens the site and may first look at the public Structures desk, which needs no account.
- The user creates an account or signs in. A new account receives $500,000 of demo cash.
- On the Trade page the user selects a cryptocurrency from the ticker strip and reads the best bid, best ask, spread and order book.
- The user chooses Buy or Sell, selects Limit or Market, and enters a quantity (and a price for a limit order).
- The API validates the request, checks that the order would not trade with the user's own resting order, reserves cash or units, and submits the order to the engine.
- The engine matches the order; each match is a trade at the resting order's price. Remaining quantity of a limit order waits in the book.
- Wallets, positions, FIFO lots, orders, trades, events and realised profit and loss are saved in one transaction.
- The screen refreshes: the order book, recent executions, open orders, balances and profit and loss all change. The Portfolio and Reports pages read the stored data.

2.10 Testing and Deployment

The engine and algorithms are tested with Node's built-in test runner, started through `tsx --test`, with test files for the data structures, algorithms, order book, edge cases and end-to-end scenarios. When the repository was run for this report, **all 41 tests passed**. (The repository README mentions 26 tests; the current test files contain 41.) The web client is built by GitHub Actions (Node 22) and published to GitHub Pages; the API is a Neon Function connected to the Neon database.

3. DATA STRUCTURES IMPLEMENTED

This chapter is the heart of the report. PriceTime uses six data structures: a binary search tree, a doubly linked list, a singly linked list, a circular linked list, a queue and a stack. Each section below gives the definition, working principle, implementation in PriceTime, where it is used, why it was selected, its advantages, its time and space complexity, and a practical example. Unless stated otherwise, the examples come from the Structures desk of the running application, using the ten-order workload shown in Chapter 7 (Ava, Ben, Cara, Dev, Eli, Fay, Gus, Hana, Ivy and Jon). Table 4 shows where each structure lives in the source code.

Table 4: *Data structures and their location in the repository (apps/api/src/engine)*

Structure	Class or field	File	Role in the engine
Binary search tree	PriceTree, PriceLevel	orderBook.ts	Sorted price levels, one tree per side
Doubly linked list	Order.prev / Order.next in PriceLevel	orderBook.ts	FIFO queue of orders at one price
Singly linked list	Trade.next, tradeHead, tradeTail	orderBook.ts	Execution tape
Circular linked list	CircularLinkedList (capacity 20)	dataStructures.ts	Rolling trade-price window
Queue	PriceLevel queue; DoublyLinkedQueue<T>	orderBook.ts, dataStructures.ts	FIFO order service; reusable queue class
Stack	Stack<OrderAction> (linked nodes)	dataStructures.ts	Audit log of submit and cancel

3.1 Binary Search Tree (BST)

Definition. A binary search tree is a rooted tree in which every node has at most two children, and for every node all keys in its left subtree are smaller and all keys in its right subtree are larger. Searching, inserting and deleting follow one path from the root, so their cost is proportional to the height of the tree.

Working principle. To find a key, compare it with the current node and move left if it is smaller or right if it is larger, until it is found or a missing child is reached. The smallest key is the leftmost node and the largest is the rightmost. Deleting a node has three cases: a leaf is simply removed, a node with one child is replaced by that child, and a node with two children is replaced by its in-order successor (the minimum of its right subtree).

Implementation in PriceTime. Class `PriceTree` stores `PriceLevel` nodes, keyed by the price in cents. Each order book owns two trees: `buyTree` for bids and `sellTree` for asks. The class provides `find`, `insert`, `minimum`, `maximum`, `delete` (all three cases, using a `transplant` helper and the in-order successor), and `levelsAscending` / `levelsDescending`, which walk the tree in order to return the top price levels for the order-book display. A node is one *price level*: it holds the price, the total quantity at that price, the number of orders and the head and tail of its queue. The tree is a plain, unbalanced BST; no rotation or colouring is implemented.

Where used. The best bid is `buyTree.maximum()` and the best ask is `sellTree.minimum()`; the matcher reads them on every loop iteration. A new limit order that does not fully trade is placed in the tree with `insert(price).enqueue(order)`. When the last order at a price leaves, `delete` removes the node, and the order-book panel is produced by an in-order walk limited to five levels.

Why selected. The book needs the best price quickly and a sorted list of levels for display, while prices keep appearing and disappearing. A hash map cannot give the maximum or the sorted order, and a sorted array makes every insertion shift elements. A BST gives all three in one structure.

Advantages. Natural sorted order; best price in one walk; insertion and deletion without shifting data; each node can carry a whole queue. **Limitation.** Because the tree is not self-balancing, prices inserted in strictly increasing or decreasing order would produce a skewed tree of height n. Real order books cluster around the market price, so the height stays small in typical use, but the worst case is O(n).

Table 5: *Complexity of the PriceTree operations (h = height, L = number of price levels, k = levels returned)*

Operation	Average	Worst	Space
find / insert price level	O(log L)	O(L)	O(1) extra
minimum / maximum (best ask / best bid)	O(log L)	O(L)	O(1)
delete level (leaf, one child, two children)	O(log L)	O(L)	O(1)
levelsAscending / levelsDescending(k)	O(h + k)	O(L)	O(h) call stack
whole tree	-	-	O(L) nodes

Practical example. After the first eight orders of the ten-order workload (that is, before Ivy and Jon trade) the bids are Eli $98, Fay $99 (and Gus $99, which joins Fay's queue), and Hana $97; the asks are Ava $101, Ben $102, Cara $100 and Dev $103. Figure 3 shows the two trees. The best bid is the rightmost node of the bid tree ($99); the best ask is the leftmost node of the ask tree ($100); the spread is $1.00. Two orders at $99 produce one node, not two.

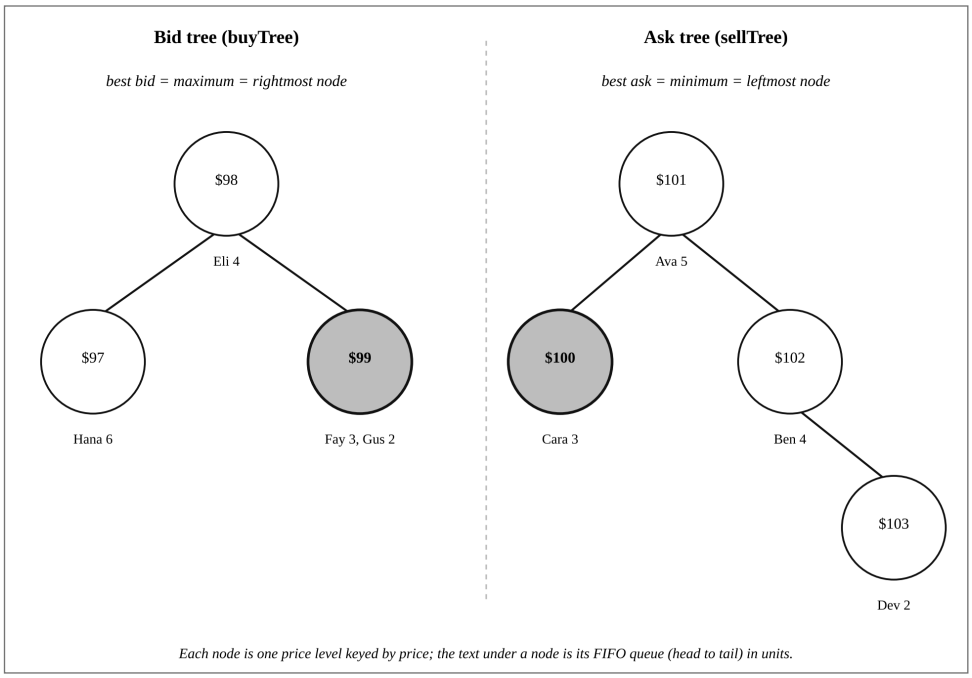


Figure 3: *Bid tree and ask tree after the first eight orders of the ten-order workload. Shaded nodes are the best bid ($99) and the best ask ($100); the text under each node is its FIFO queue.*

3.2 Doubly Linked List (DLL)

Definition. A doubly linked list is a sequence of nodes in which each node stores a value and two references, one to the previous node and one to the next. It can be traversed in both directions and, given a node, that node can be removed without searching for it.

Working principle. Adding at the tail makes the old tail point forward to the new node and the new node point back to the old tail. Removing a node makes its predecessor skip forward over it and its successor skip back over it; if the node was the head or tail, the head or tail reference moves. Both operations touch a fixed number of references, so both are O(1).

Implementation in PriceTime. The orders at one price are linked through the fields `prev` and `next` of each `Order`. The owning `PriceLevel` keeps `head` and `tail`. Method `enqueue(order)` appends at the tail and adds the quantity to the level total; method `unlink(order)` removes any order in O(1), fixes the neighbours, subtracts the quantity, and clears the order's own pointers. Each order also stores a reference to its level, so the level can be found directly.

Where used. The head of the list is always the order that has waited longest at that price, so the matcher fills `opposite.head` first (time priority). When an order is cancelled, `order.level.unlink(order)` removes it from the middle of its queue without scanning; if the level becomes empty, the tree node is deleted.

Why selected. The queue at a price must support append at the back, serve from the front, and cancel from anywhere. A singly linked list or an array would need a scan to cancel, and removing from the middle of an array shifts all later elements. The doubly linked list does all three in constant time.

Advantages. O(1) enqueue, O(1) removal of a known node, two-way traversal, no shifting, no capacity limit. **Limitation.** Two references per node use more memory than a singly linked list, and finding an order by value still needs a search (which is why PriceTime keeps orders in a sorted array for lookup by id).

Table 6: *Complexity of the doubly linked order queue (n = orders at that price)*

Operation	Time	Space
enqueue at tail	O(1)	O(1) per order
serve (read) the head	O(1)	O(1)
unlink a known order (cancel or fill)	O(1)	O(1)
traverse the queue (display)	O(n)	O(1)

Practical example. On the ten-order workload Fay (3 units) and Gus (2 units) both bid $99, Fay first. When Ivy sells 4 units at $99, the engine serves Fay's order first (3 units), unlinks it, and then takes 1 unit from Gus, who stays at the head with 1 unit left (Figure 4). In Figure 5, a cancel of the middle order B only rewrites A.next and C.prev.

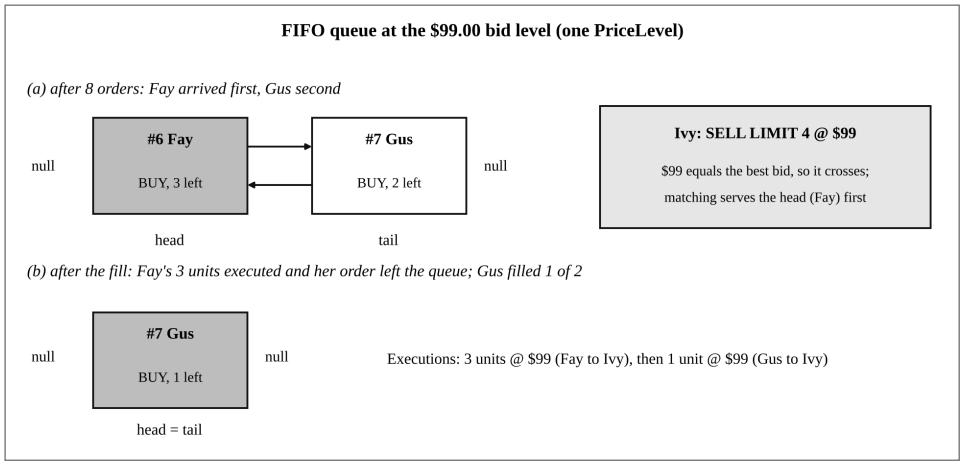


Figure 4: *FIFO queue at the $99.00 bid level before and after Ivy's sell order of 4 units.*

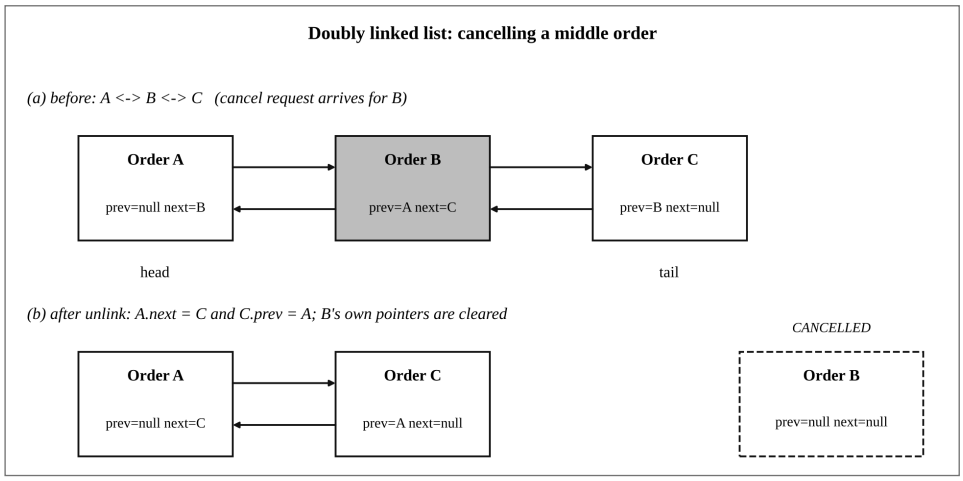


Figure 5: *Cancelling an order from the middle of a doubly linked queue in O(1).*

3.3 Singly Linked List (SLL)

Definition. A singly linked list is a chain of nodes, each holding a value and a reference to the next node only. It is traversed in one direction, from the head to the end.

Working principle. If the list keeps both a head and a tail reference, appending is O(1): the old tail's `next` is set to the new node and the tail moves. Reading all elements walks from the head.

Implementation in PriceTime. Every executed trade is a `Trade` object with a `next` field. The order book keeps `tradeHead` and `tradeTail`; method `appendTrade` links the new trade after the tail and also adds its price to the circular window. Method `trades()` walks the chain from the head to rebuild the full tape in time order; the snapshot takes the last 20 and reverses them for the Recent executions panel.

Where used. The execution tape: the permanent, time-ordered record of trades in a book. It is read by the snapshot, by the Structures desk, and by the last-price calculation (`tradeTail.pricePaise`).

Why selected. Trades are only ever added at the end and read in order; they are never removed from the middle. A singly linked list with a tail pointer is the simplest structure that fits, and it uses one reference per node instead of two. **Advantages:** O(1) append, simple, no resizing. **Limitation:** reaching the last 20 trades requires walking the whole chain, O(T).

Table 7: *Complexity of the singly linked execution tape (T = trades recorded)*

Operation	Time	Space
appendTrade (tail pointer)	O(1)	O(1) per trade
last trade price (tail)	O(1)	O(1)
trades() / full traversal	O(T)	O(T) for the output array

Practical example. On the ten-order workload the tape holds four trades: 3 units at $99 (buyer Fay, seller Ivy), 1 unit at $99 (buyer Gus, seller Ivy), 3 units at $100 (buyer Jon, seller Cara) and 1 unit at $101 (buyer Jon, seller Ava), as drawn in Figure 6. Each label follows the application's *Buyer to seller* column, so the first name is the buyer.

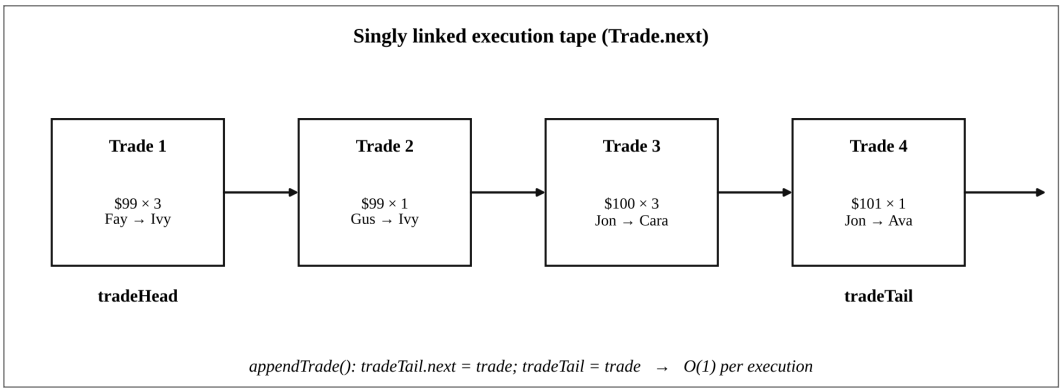


Figure 6: *Execution tape as a singly linked list; each trade points to the next one.*

3.4 Circular Linked List (CLL)

Definition. A circular linked list is a linked list in which the last node points back to the first, so there is no null end. Traversal can continue around the ring indefinitely.

Working principle. While the ring has room, a new node is added after the tail and made to point to the head. When the ring is full, the oldest node (the head) is overwritten with the new value and the head and tail references each move one step forward; no node is created or destroyed.

Implementation in PriceTime. Class `CircularLinkedList<T>` has a fixed capacity (20 in the order book), a head, a tail and a size, and keeps the invariant `tail.next === head`. Method `append` implements the rule above, `toArray` returns values from oldest to newest, and `average` adds the values in the ring and divides by the number of stored prices, rounding to a whole cent.

Where used. `recentPrices` in each order book: `appendTrade` adds every execution price to it, and `movingAverage()` supplies the 20-trade average shown above the Recent executions table and in the order-book snapshot.

Why selected. The application needs only the most recent 20 prices. A ring has constant memory, never grows, and drops the oldest value automatically, so no separate deletion code is needed. **Advantages:** bounded memory, O(1) append, natural fit for a sliding window. **Limitation:** older prices are lost, and the average is recomputed by walking at most 20 nodes.

Table 8: *Complexity of the circular price window (c = capacity = 20)*

Operation	Time	Space
append (new node or overwrite oldest)	O(1)	O(c) in total
average() / toArray()	O(c), at most 20 steps	O(1) / O(c)

Practical example. After the four executions of the ten-order workload the ring contains 99, 99, 100 and 101, so the average is (99 + 99 + 100 + 101) / 4 = $99.75 (Figure 7, part a). After the ring holds 20 prices, the 21st execution overwrites the oldest value (part b, drawn with capacity 6 for clarity).

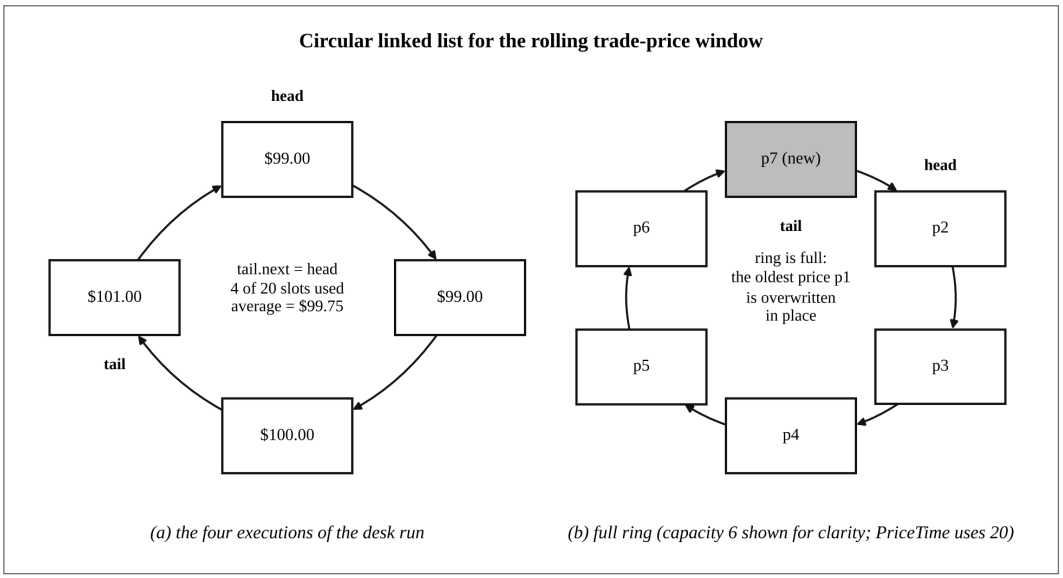


Figure 7: *Circular linked list used as a rolling window of trade prices.*

3.5 Queue

Definition. A queue is a first-in, first-out (FIFO) collection: elements are added at the back and removed from the front, so the element that has waited longest is served first.

Working principle. Two operations define a queue: *enqueue* (add at the back) and *dequeue* (remove from the front), plus *peek* to read the front without removing it. A linked implementation with head and tail references makes all three O(1).

Implementation in PriceTime. The FIFO queue of one price level is the doubly linked list of Section 3.2; the level's `head` is the front and `tail` is the back. The file `dataStructures.ts` also contains a general class `DoublyLinkedQueue<T>` (enqueue, dequeue, peek, unlink); it is unit-tested, but the order book itself uses the node pointers inside `PriceLevel`.

Where used. The queue is what turns a price level into *time priority*. Two orders at the same price are ordered only by arrival, never by size or by user; the Structures desk shows a price's queue when its tree node is clicked.

Why selected. Fairness among equal prices is exactly the FIFO rule. **Advantages:** O(1) enqueue and service; simple to reason about; combined with the doubly linked pointers it also supports cancel. **Limitation:** a plain queue gives no access by value; cancellation relies on the extra prev pointer.

Table 9: *Complexity of queue operations*

Operation	Time	Space
enqueue	O(1)	O(1) per element
dequeue / serve head	O(1)	O(1)
peek	O(1)	O(1)
unlink a known node	O(1)	O(1)

Practical example. On the five-order sample of the Structures desk, Cara and Dev both bid $99. Cara arrived first, so she is the head of the $99 queue and is filled before Dev.

3.6 Stack

Definition. A stack is a last-in, first-out (LIFO) collection. Elements are pushed onto and popped from the same end, called the top, so the most recently added element is the first to leave.

Working principle. *push* creates a node whose next reference is the current top and makes it the new top; *pop* returns the top value and moves the top to the next node; *peek* reads the top without removing it.

Implementation in PriceTime. Class `Stack<T>` is built from `StackNode` objects (push, pop, peek, toArray). Each order book owns a `Stack<OrderAction>`. An `OrderAction` records the kind (SUBMIT or CANCEL), order id, symbol, side, type, price, quantity and time.

Where used. `submit()` pushes a SUBMIT action and `cancel()` pushes a CANCEL action. The engine reads it with `peek()` and `toArray()`, and the Structures desk draws it as the Action stack panel. The `pop()` method is implemented and tested but not called by the engine, because the audit log is never undone.

Why selected. The question 'what did the engine just do?' is a last-in question, and a stack answers it in O(1). **Advantages:** constant-time operations, trivial implementation, no shifting; only the top is directly reachable.

Table 10: *Complexity of the action stack (n = actions)*

Operation	Time	Space
push / pop / peek	O(1)	O(1)
toArray (history); whole stack uses O(n) nodes	O(n)	O(n)

Practical example. After ten orders the top is *SUBMIT #10 BUY*, then *SUBMIT #9 SELL $99.00* (Figure 8).

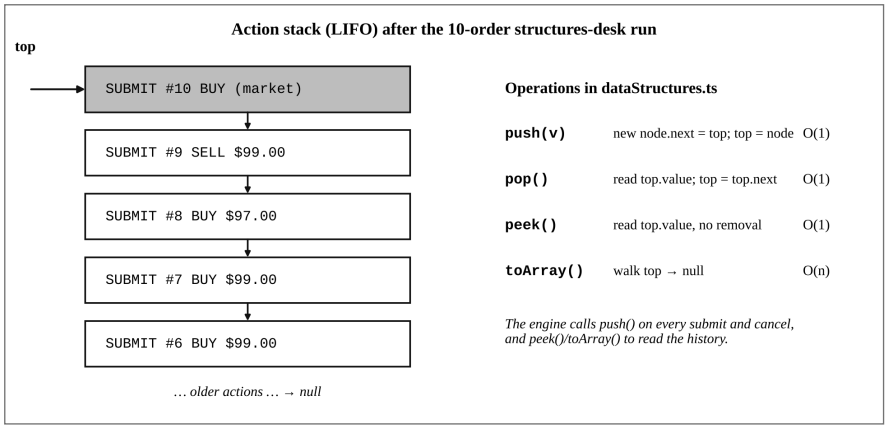


Figure 8: *Action stack after the ten-order workload; the latest action is on top.*

4. ALGORITHMS

PriceTime uses two searching algorithms and five sorting algorithms. The web client contains the versions used by the screens (`apps/web/src/algorithms/search.ts` and `sorts.ts`); the API engine contains the same family in `apps/api/src/engine/algorithms.ts`, and its `binarySearch` is used inside the order book. None of the sorts calls the built-in `Array.prototype.sort`. In the formulas below n is the number of elements.

4.1 Searching Algorithms

Linear search

Definition and principle. Linear search checks the elements one after another from the first to the last and keeps those that satisfy a condition. It needs no ordering.

Steps. (1) Start at index 0. (2) Test the element against the condition. (3) If it matches, record it. (4) Move to the next index; stop after the last element.

Application in PriceTime. The *Linear search* box on the Reports page filters executions by trader, symbol or trade id as the user types; the function `linearSearch` returns every match. A first-match variant, `linearSearchFirst`, is written and tested but is not used by the screens.

Binary search

Definition and principle. Binary search finds a key in a *sorted* array by comparing it with the middle element and discarding the half that cannot contain it. The range halves at each step.

Steps. (1) Set low = 0 and high = n - 1. (2) While low is not greater than high, compute mid = floor((low + high) / 2). (3) If the key at mid equals the target, return it. (4) If it is smaller than the target, set low = mid + 1; otherwise set high = mid - 1. (5) If the range becomes empty, the key is absent.

Application in PriceTime. (a) Inside the engine, `findOrder(id)` binary-searches the order array, which stays sorted because order ids only increase; it is used by `cancel`. (b) On the Reports page the *Binary search by trade id* box first orders the executions by trade id (with insertion sort) and then searches them, printing the probes (for example, '#2 too small, #4 found'). Trade ids are numbered per symbol, so the same id can occur for different symbols; the search reports the first match it reaches. (c) On the Structures desk the order-table lookup shows each probe. (d) The engine's `submit()` keeps its order array sorted by inserting each new order with a scan, which costs O(n); only the lookup is logarithmic.

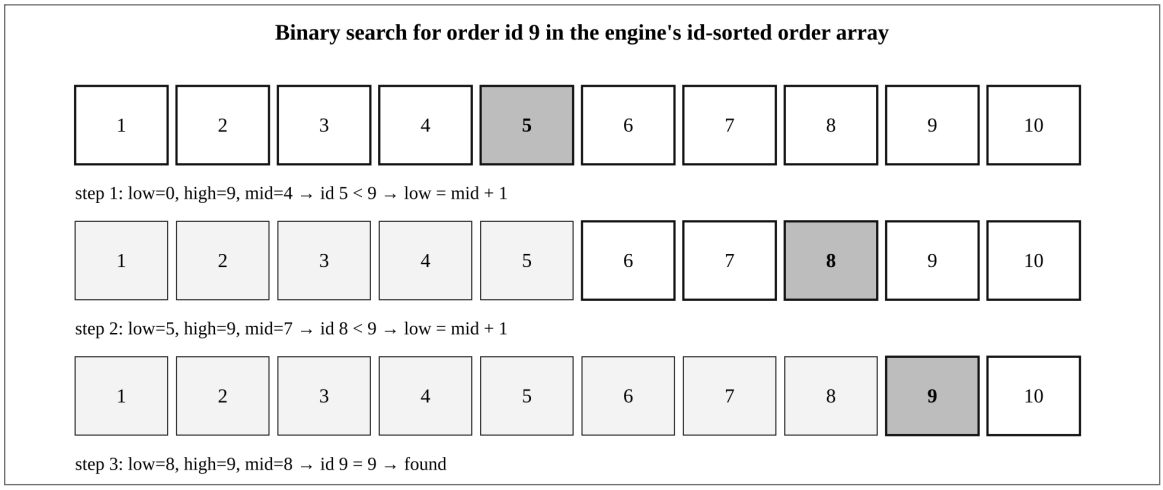


Figure 9: *Binary search for order id 9 in a sorted array of ten orders; three probes (ids 5, 8, 9).*

Table 11: *Comparison of the two searching algorithms*

Algorithm	Best	Average	Worst	Space	Needs sorted data
Linear search	O(1)	O(n)	O(n)	O(1) (O(m) for m matches)	No
Binary search	O(1)	O(log n)	O(log n)	O(1)	Yes

Example. In Figure 9, a search for id 9 among ten ordered ids needs three comparisons (ids 5, 8, 9). A linear scan of the same array would need nine.

4.2 Sorting Algorithms

The Reports page and the Structures desk let the user choose a sort algorithm (bubble, selection, insertion, merge or quick) and a field (time, price, quantity and so on), then sort a *copy* of the data and show the number of records and the elapsed time. Choosing a different algorithm gives the same ordering; only the method differs.

Bubble sort

Definition and principle. Repeatedly compare adjacent elements and swap them if they are in the wrong order; after each pass the largest remaining element 'bubbles' to the end. **Steps.** For each pass, compare items i and i+1 up to the unsorted end; swap if needed; stop early if a pass makes no swap. **In PriceTime.** Implemented with an early-exit flag, so an already sorted list takes one pass.

Selection sort

Definition and principle. Repeatedly select the smallest element of the unsorted part and place it at the front of that part. **Steps.** For each position i, scan the rest to find the minimum, swap it into position i. **In PriceTime.** Does at most n - 1 swaps, but always performs all comparisons.

Insertion sort

Definition and principle. Build the sorted part one element at a time: take the next element and shift larger elements right until its place is found. **Steps.** For i from 1, store items[i], move left while the previous item is larger, drop the stored item into the gap. **In PriceTime.** Efficient for short or almost sorted lists, which is typical for a user's recent executions.

Merge sort

Definition and principle. A divide-and-conquer method: split the list into halves, sort each half recursively, and merge the two sorted halves. **Steps.** If the length is below 2 return it; split at the middle; sort both halves; merge by repeatedly taking the smaller front element (taking the left one on a tie). **In PriceTime.** The merge takes the left element when the keys are equal, so the sort is **stable**; this matters when sorting trades by price, because trades at the same price stay in time order.

Quick sort

Definition and principle. Choose a pivot, rearrange the list so that smaller elements are on its left and larger on its right, then sort both parts recursively. **Steps.** Take the middle element as pivot; move indices i from the left and j from the right toward each other, swapping out-of-place pairs (Hoare-style partition); recurse on the two parts. **In PriceTime.** Sorts in place with the middle element as pivot, which avoids the worst case on already sorted input; it is not stable.

Table 12: *Comparison of the five sorting algorithms*

Algorithm	Best	Average	Worst	Space	Stable
Bubble sort (early exit)	O(n)	O(n²)	O(n²)	O(1)	Yes
Selection sort	O(n²)	O(n²)	O(n²)	O(1)	No
Insertion sort	O(n)	O(n²)	O(n²)	O(1)	Yes
Merge sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes
Quick sort (middle pivot)	O(n log n)	O(n log n)	O(n²)	O(log n) stack	No

Which algorithm suits which data. For the short lists of a classroom account (a handful of executions) all five are instant, and the screens exist so that students can compare behaviour. For large lists, merge sort and quick sort scale as n log n while the three simple sorts grow as n squared. In practice, insertion sort suits very short or nearly sorted lists, merge sort is the choice when equal prices must keep their time order, and quick sort suits large lists when memory is tight.

4.3 Use of These Algorithms in the Reports Section

On the Reports page the user can (1) type in the linear-search box to filter the executions, (2) choose a sort field (time, price or quantity) and an algorithm and press *Sort report*, and (3) enter a trade id for binary search. The status line shows the records sorted, the time taken in milliseconds, and the binary-search probes. All of this runs in the browser on the loaded data.

4.4 FIFO Accounting for Realised Profit and Loss

Definition. FIFO accounting treats the earliest-bought units as the first ones sold (the queue rule applied to inventory).

Implementation in PriceTime. Each buy creates a *lot* (quantity and unit cost in cents) in `mf_portfolio_lots`. When a sell trade is settled, `settleTrade` reads the seller's lots for that symbol ordered by opening time and id, consumes the oldest first, and records `cost basis`, `proceeds` and `realized = proceeds - cost basis` in `mf_realized_pnl`. The remainder of a partly used lot stays open. *Unrealised* profit and loss is the last trade price times the units held, minus the cost basis of the remaining lots.

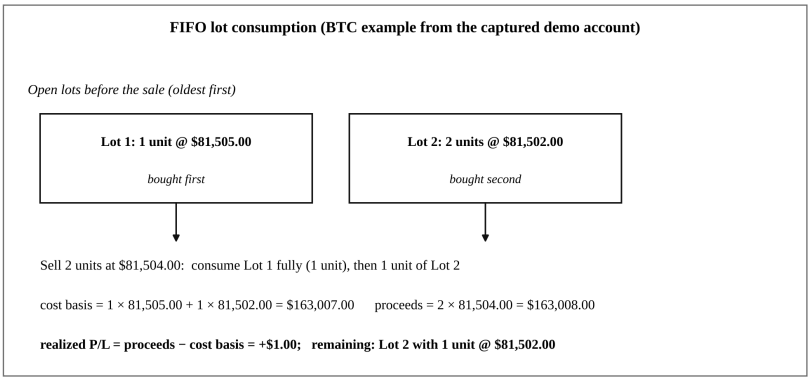


Figure 10: *FIFO lot consumption for one sale in the captured BTC demo account: realised profit and loss of +$1.00.*

Example. The demo account holds Lot 1 (1 unit at $81,505) and Lot 2 (2 units at $81,502). Selling 2 units at $81,504 uses all of Lot 1 and 1 unit of Lot 2: cost basis $163,007, proceeds $163,008, realised profit +$1.00. One unit of Lot 2 remains (1 BTC at $81,502.00 in the Portfolio).

5. PRICE-TIME MATCHING ENGINE AND SYSTEM WORKFLOW

5.1 The Order Book

An **order book** is the list of orders that are waiting to be filled for one asset. Buy orders (**bids**) are listed from the highest price down, and sell orders (**asks**) from the lowest price up. The highest bid is the **best bid**, the lowest ask is the **best ask**, and the difference between them is the **spread**. PriceTime keeps one book per symbol (seven in total). Within a book, orders at the same price are grouped into one price level; the level's orders wait in arrival order.

5.2 Order Types and Rules

A **limit order** says: buy no higher than, or sell no lower than, this price. If it crosses the opposite best price it trades at once; any part that cannot trade waits in the book. A **market order** says: trade now at whatever prices are available. It sweeps the best price level, then the next, until it is filled or the opposite side is empty; it never rests in the book. Table 13 lists the rules implemented by the engine.

Table 13: *Matching rules implemented in PriceTime*

Rule	How PriceTime applies it
Price priority	The best price is always served first: highest bid for a seller, lowest ask for a buyer (maximum of the bid tree, minimum of the ask tree).
Time priority (FIFO)	At one price, the order at the head of the doubly linked queue (oldest) is served first.
Crossing test	A buy limit order crosses when its price is at least the best ask; a sell limit order crosses when its price is at most the best bid.
Execution price	A trade is made at the price of the *resting* order, so an incoming order can receive a better price than it asked for (price improvement).
Partial fill	A trade quantity is the smaller of the two remaining quantities. The larger order stays PARTIALLY_FILLED and keeps its place (a resting order stays at the head of its queue).
Market order with no liquidity	If nothing was filled the order is REJECTED. If it was partly filled, the order ends PARTIALLY_FILLED and the unfilled remainder is released (it does not rest).
Cancellation	Only OPEN or PARTIALLY_FILLED orders can be cancelled. The order is unlinked from its queue in O(1), an empty level is removed from the tree, and reserved cash or units are returned.
Self-trade prevention	Before an order reaches the engine, the API checks the user's own resting orders on the opposite side. If the new order would cross any of them, it is refused with HTTP 409: 'Your order would match against your own open order. Cancel or amend it first.'
Funds check	A buy reserves cash (limit price times quantity, or the estimated cost for a market buy); a sell reserves units. Not enough cash or units gives HTTP 422.
Order status	OPEN, PARTIALLY_FILLED, FILLED, CANCELLED or REJECTED.

5.3 How the Binary Search Tree and the Doubly Linked List Work Together

The two structures divide the work of price-time priority. The **tree** answers 'which price?' and the **list** answers 'which order at that price?'. When a buy order arrives, the engine takes `sellTree.minimum()`, which is the best ask level. If the buy price is at least that price, the engine reads `level.head`, the oldest order at that level, trades the smaller of the two remaining quantities, and appends a trade to the singly linked tape. If the head order is exhausted it is unlinked from the list; if the list becomes empty the level is deleted from the tree, and the loop continues with the next-best price. If the incoming order still has quantity left and is a limit order, `buyTree.insert(price)` finds or creates its level and `enqueue` adds it at the tail of that level's queue. Every step is therefore a small, local pointer change.

5.4 Worked Example

The example uses the ten-order workload entered on the Structures desk (see Chapter 7). After the first eight orders, the book is as in Figure 11(a): the best ask is $100.00 (Cara, 3 units), the best bid is $99.00 with Fay (3 units) in front of Gus (2 units), and the spread is $1.00. Two more orders then arrive.

Table 14: *Matching steps for orders #9 and #10*

Order	Matching steps	Result
#9 Ivy, SELL LIMIT 4 @ $99	A sell crosses when its price is at most the best bid; $99 equals the best bid. The head of the $99 queue is Fay (3 units): trade 3 @ $99; Fay is filled and unlinked. Next is Gus (2 units): trade 1 @ $99; Gus keeps 1 unit.	Ivy FILLED. Gus PARTIALLY_FILLED (1 left). Two trades.
#10 Jon, BUY MARKET 4	A market order accepts any price. The best ask is $100 (Cara, 3): trade 3 @ $100; Cara is filled, and the $100 level is deleted from the ask tree. The new best ask is $101 (Ava, 5): trade 1 @ $101; Ava keeps 4.	Jon FILLED. Ava PARTIALLY_FILLED (4 left). Two trades.

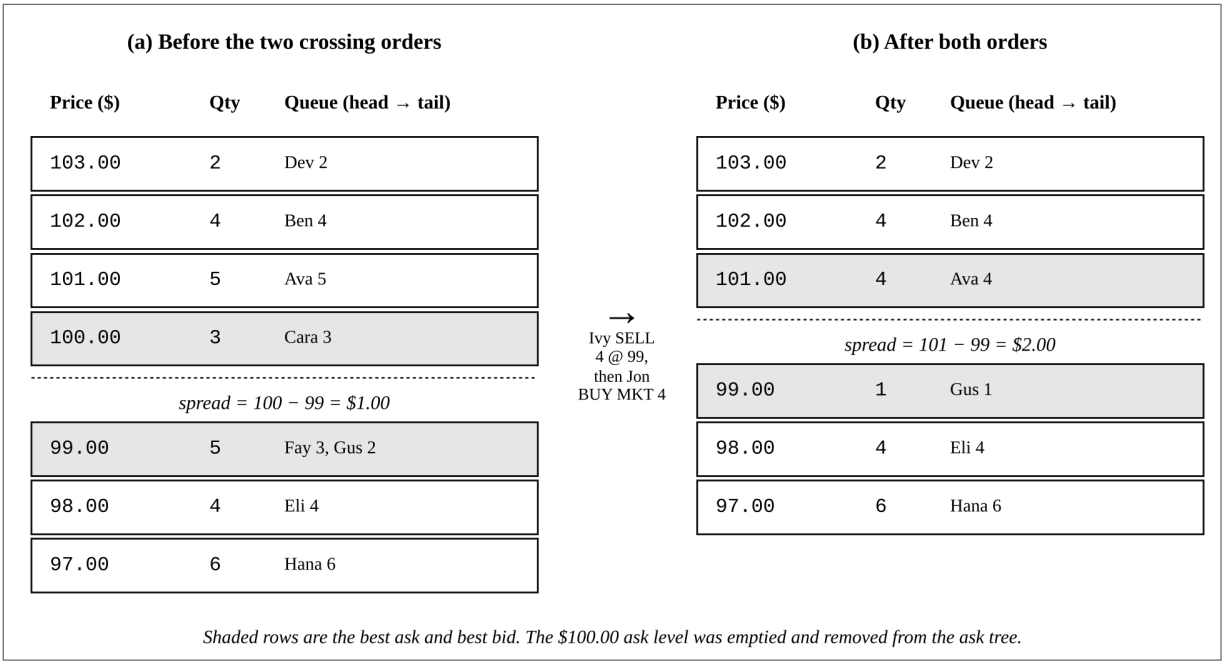


Figure 11: *Order book of the worked example before (a) and after (b) orders #9 and #10. Shaded rows are the best ask and best bid.*

After both orders the book has best bid $99.00 (Gus, 1 unit), best ask $101.00 (Ava, 4 units) and a spread of $2.00. Four trades were recorded (3 @ $99, 1 @ $99, 3 @ $100, 1 @ $101), and the 20-trade average is (99 + 99 + 100 + 101) / 4 = $99.75. The Structures desk screenshots in Chapter 7 show exactly these values.

5.5 System Workflow Inside the API

For a real order the API does more than call the engine. In one database transaction it (1) takes the advisory lock so that only one order is processed at a time; (2) loads the book by replaying the stored order events in sequence order; (3) rejects a self-trade; (4) checks and reserves cash or units; (5) writes the order and a *submit* event; (6) calls `book.submit`; (7) settles each trade, which moves cash and units between the two participants, creates a FIFO lot for the buyer, consumes the seller's lots and records realised profit and loss; (8) updates the status of every order that changed and inserts the trades; and (9) commits. If any step fails, the whole transaction is rolled back and the user sees 'Order could not be completed. No account changes were saved.'

PriceTime is not a real exchange. It is a single-process teaching engine. It has no network of participants, no clearing house, no regulatory reporting and no connection to a real market. The seeded market-maker account (`paper-market-maker`) posts 50-unit bids and asks per symbol so that the simulated books are not empty. Real exchanges use very different engineering (for example self-balancing or array-based price ladders, in-memory sequencers and replicated logs); the purpose of PriceTime is to show the logic of price-time priority with the structures taught in the course.

6. TECHNOLOGIES, APPLICATIONS AND FUTURE SCOPE

6.1 Technologies Used

Table 15 lists only the technologies that appear in the repository's package files, source code or deployment workflow. Version numbers are the ranges declared in the package files.

Table 15: *Technologies verified in the PriceTime repository*

Layer	Technology	Use in PriceTime
Client	React 18 (^18.3.1), TypeScript (^5.7.2), Vite (^6.0.5)	Single-page web application and build tool.
Client	lucide-react (^0.468.0)	Interface icons.
Client	TradingView Advanced Chart widget	Embedded price chart for BINANCE:XXXUSDT (display only).
Client	Binance public ticker data	Reference prices shown in the ticker strip (display only; no orders are sent).
Authentication	Neon Auth (@neondatabase/auth), JWT	Email and password sign-in; signed token sent to the API.
Backend	Node.js, Hono (^4.13.13), Neon Functions (@neon/functions)	REST API and its hosting.
Backend	jose (^6.2.12), Zod (^3.24.2)	Token verification against JWKS; request validation.
Database	Neon PostgreSQL, node-postgres 'pg' (^8.23.1)	Transactions, advisory lock, storage of all records.
Database	Drizzle ORM (^0.45.4), drizzle-kit (^0.31.11)	Typed schema (nine tables) and SQL migrations.
Testing	tsx (^4.19.2) with Node's test runner (node:test)	41 automated tests for the engine and algorithms.
Deployment	GitHub Pages, GitHub Actions (Node 22)	Build and hosting of the static web client.

Note: the client's package file also lists `socket.io-client`, but no source file imports it; PriceTime does not currently use WebSockets, and the screens refresh by requesting data from the API.

6.2 Applications

- **Trading education.** A beginner can learn what bids, asks, spreads, limit and market orders mean, with demo funds and no financial risk.
- **Teaching data structures.** The Structures desk shows trees, queues, linked lists, rings and stacks changing after every order, which links classroom theory to a working system.
- **Practice of trading ideas.** Users can test how order type and price affect fills, partial fills and profit and loss, using FIFO accounting.
- **Demonstration of market mechanics.** Instructors can show price-time priority, partial fills, cancellations and self-trade prevention with concrete examples.
- **Algorithm comparison.** The Reports page and the CSV replay let students compare searching and sorting algorithms and compare the tree-and-queue engine with a plain-array baseline on the same

orders.

- **Base for further projects.** The typed code, schema and tests can serve as a starting point for coursework or portfolio projects in back-end development and fintech.

6.3 Limitations of the Present Version

To keep the report accurate, the following limits of the current implementation are stated. The price tree is not self-balancing, so its worst case is O(n). Every request rebuilds the book by replaying the stored events, and a single database lock serialises all order processing, which is acceptable for a classroom load but would not scale to heavy traffic. Quantities are whole numbers of units. Self-trade prevention is performed in the API layer rather than inside the engine. The circular list keeps only the last 20 prices, and the action stack is a read-only log (its pop method is never called). These are design trade-offs of an educational project, not defects of the matching rules.

6.4 Future Scope (proposed, not yet implemented)

The ideas below are **proposals only**. None of them exists in the repository today.

- Replace the plain BST with a self-balancing tree (AVL or red-black) to guarantee O(log n) in the worst case.
- Add more order types, such as stop, stop-limit, immediate-or-cancel and fill-or-kill, and the ability to amend an order.
- Push live updates to the browser with WebSockets instead of requesting data again.
- Store a snapshot of each book so that the event log does not have to be replayed in full on every request.
- Add leaderboards, profit and loss charts over time, and multi-user trading competitions for a class.
- Replay recorded real market data through the engine and extend the benchmark to larger and more varied workloads.
- Support fractional quantities and more cryptocurrencies.

7. PRACTICAL IMPLEMENTATION

This chapter shows PriceTime in use. **Source of the screenshots.** The live site (sonstellar969.github.io/PriceTime) and its cloud services (Neon Auth, Neon database, TradingView and Binance) could not be reached from the environment in which this report was prepared.

To obtain genuine screenshots, the repository code was therefore run on a local machine: the unmodified API (`function.ts`) and the unmodified web client were started in a local development build, connected to a local PostgreSQL database with the repository's schema, and a small local stand-in replaced only the sign-in provider so that a demo user could log in. All orders, trades, balances and structure views in the figures were produced by the project's own code. Nothing in the figures was edited or invented. The one visual that could not be captured is the TradingView chart (a labelled placeholder is shown in Section 7.2). Because Binance was also unreachable, the ticker strip shows the application's built-in reference prices (for example BTC $81,500.00) instead of live Binance prices.

7.1 Sign-in and the Public Structures Desk

A visitor first sees the sign-in screen (Figure 12). New accounts receive $500,000 of demo cash, and a link leads to the Structures desk, which needs no account.

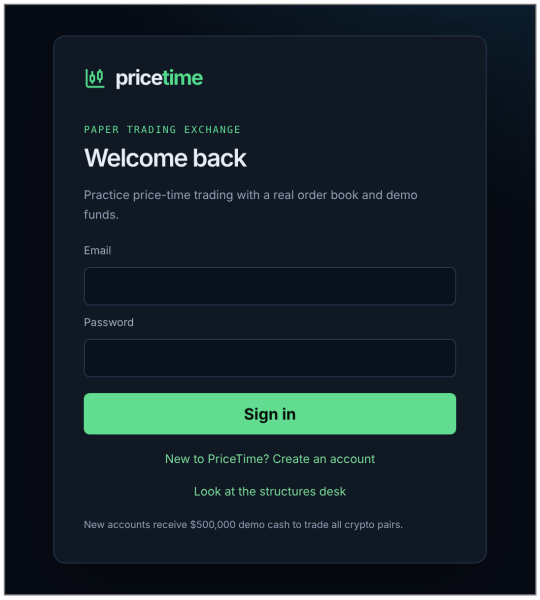


Figure 12: *Sign-in screen: email and password, a link to create an account, and a link to the Structures desk (local development build).*

The Structures desk runs the same `OrderBook` class as the server, in the browser. Figure 13 shows the workload table: each row is an order (trader, side, type, price, quantity), and a slider chooses how many rows have been applied. The ten orders shown are the ones used for the examples in Chapters 3 to 5.

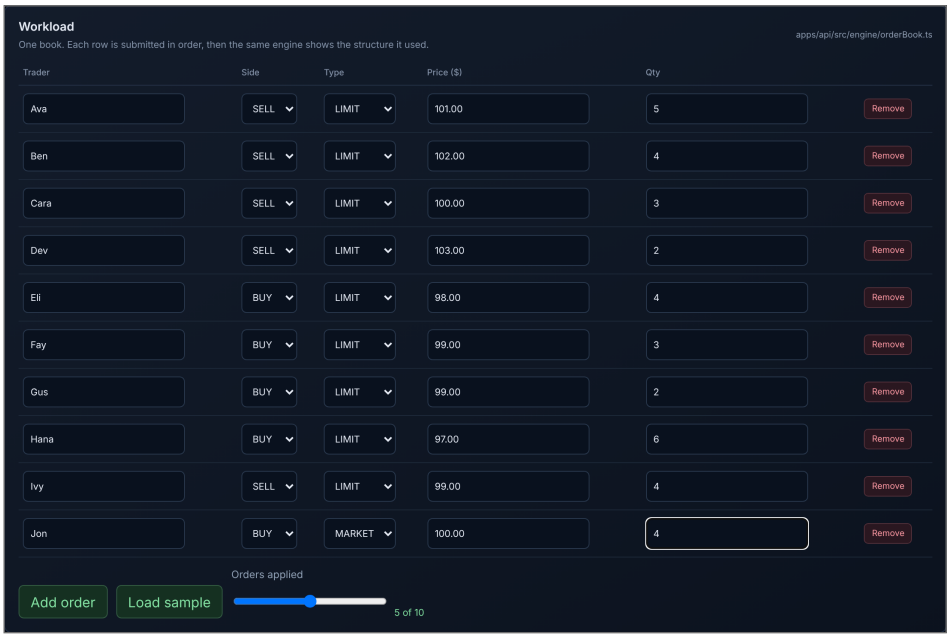


Figure 13: *Structures desk workload: ten orders (Ava to Jon), with Add order, Load sample and the 'Orders applied' slider.*

With all ten orders applied, the panels in Figure 14 and Figure 15 show the state of every structure. The key figures read best bid $99.00, best ask $101.00, spread $2.00, 20-trade average $99.75 and four executions. The price trees show the bid and ask trees; the queue at $99.00 shows Gus with 1 unit left as head; the ring holds the four trade prices; the action stack shows SUBMIT #10 BUY on top; the tape lists the four trades; and the order table was searched for order 9 by binary search, with probes at #5, #8 and #9, and sorted by price with quick sort.



Figure 14: *Key figures of the Structures desk after ten orders.*

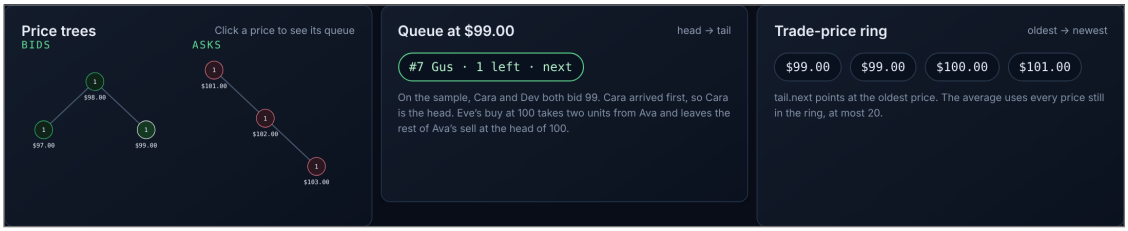


Figure 15: *Price trees, queue at $99.00 (head to tail) and trade-price ring.*

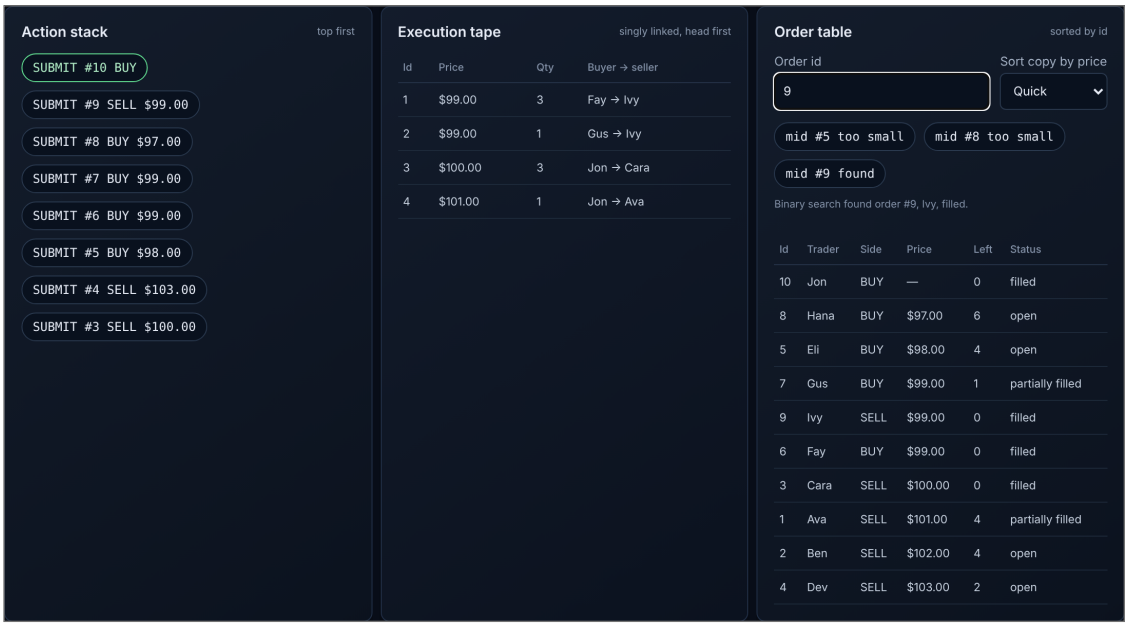


Figure 16: *Action stack, execution tape (singly linked) and order table with binary-search probes and a sorted copy.*

7.2 Trade Desk

After signing in as the demo user, the Trade page opens. The ticker strip shows the seven cryptocurrencies with their reference prices, and the key figures show the last simulated trade, best bid, best ask, spread, available demo cash, realised FIFO profit and loss, unrealised profit and loss and the holding in the selected symbol (Figure 17).

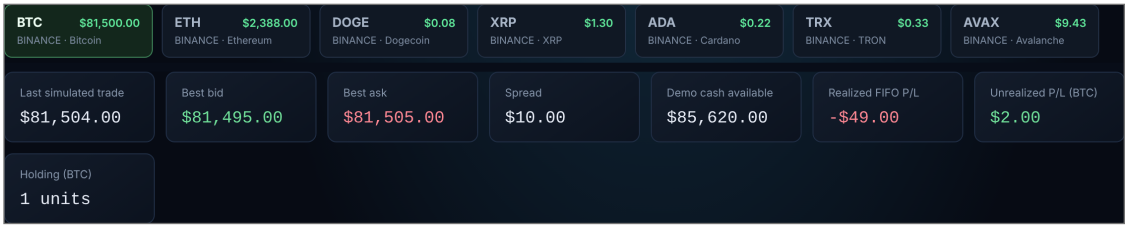


Figure 17: *Trade desk header: ticker strip (built-in reference prices, Binance unreachable) and key figures for BTC after the demo trading session.*

SCREENSHOT NOT CAPTURED

Trade desk chart panel (TradingView Advanced Chart for BINANCE:BTCUSDT)

Screenshot not captured. The chart is loaded by the browser from tradingview.com, which was not reachable from the environment used to prepare this report, and its content is not produced by PriceTime code. A genuine screenshot should be taken from the live site (https://sonstellar969.github.io/PriceTime/) and inserted here.

Figure 18: *Trade desk chart panel (TradingView Advanced Chart for BINANCE:BTCUSDT) (not captured)*

The order ticket (Figure 19) lets the user choose Buy or Sell, Limit or Market, a quantity and a limit price; it shows the available cash and the reminder that fills are simulated. The order book (Figure 20) lists the

top five levels aggregated on each side, with the spread between them, and the Recent executions table (Figure 21) shows the last trades with the 20-trade average. The values in these panels are produced by the engine from the demo orders.

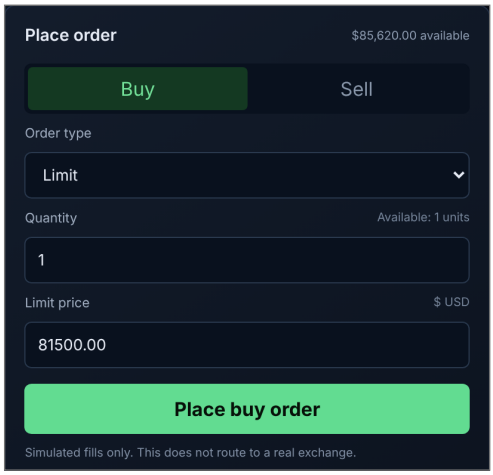


Figure 19: *Order ticket: side, order type, quantity, limit price and the simulated-fills notice.*

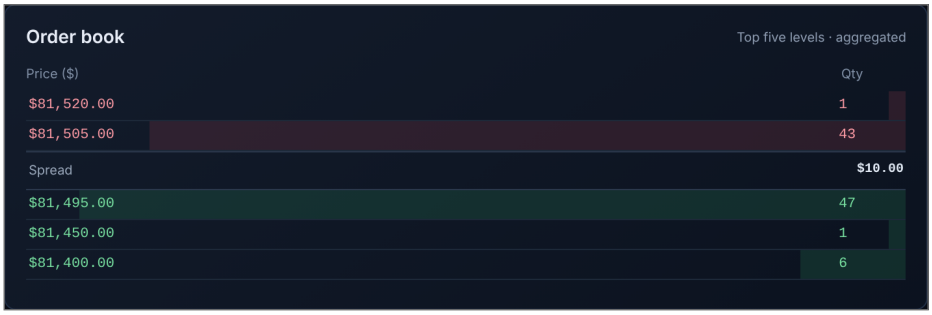


Figure 20: *Order book panel: top five levels per side, quantity bars and the $10.00 spread.*

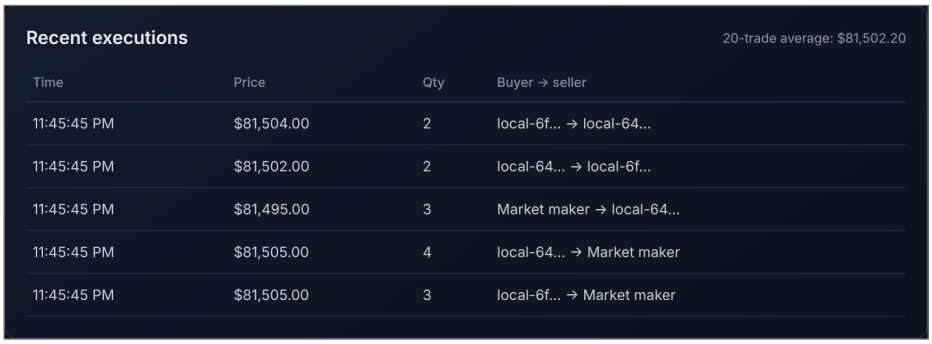


Figure 21: *Recent executions and the 20-trade average ($81,502.20).*

Open orders (Figure 22) can be cancelled with one click. Figure 23 shows three messages produced by real requests: a sell market order that would trade against the demo user's own resting bids is refused by self-trade prevention (HTTP 409); a limit buy for 1 unit at $81,300.00 is accepted and rests in the book; and cancelling it returns the reserved funds. Figure 24 shows the order book after the limit order was placed, with the new $81,300.00 level at the bottom of the bids.



Figure 22: *My open orders: two resting buy orders with Cancel buttons.*

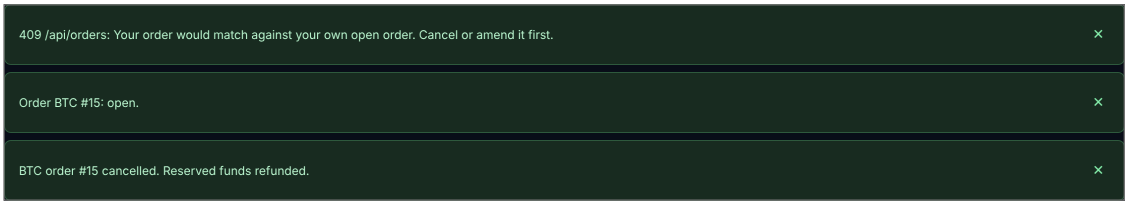


Figure 23: *Self-trade prevention (409), order accepted, and cancellation with refund.*

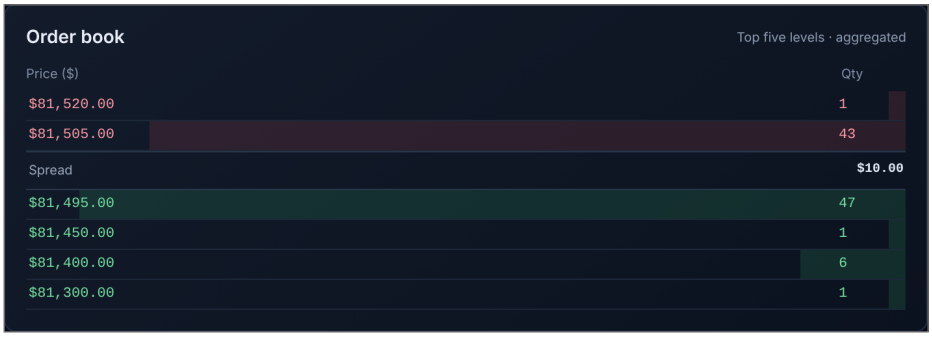


Figure 24: *Order book after the $81,300.00 bid was added (quantity 1 at the lowest bid level).*

7.3 Portfolio

The Portfolio page (Figure 25) shows available cash ($85,620.00), realised FIFO profit and loss (-$49.00), unrealised profit and loss (-$28.00) and the net total (-$77.00), followed by a table of positions with units, available and reserved quantity, average cost, last trade price and unrealised profit and loss per symbol. BTC shows 1 unit at an average cost of $81,502.00, which matches the FIFO example in Section 4.4.

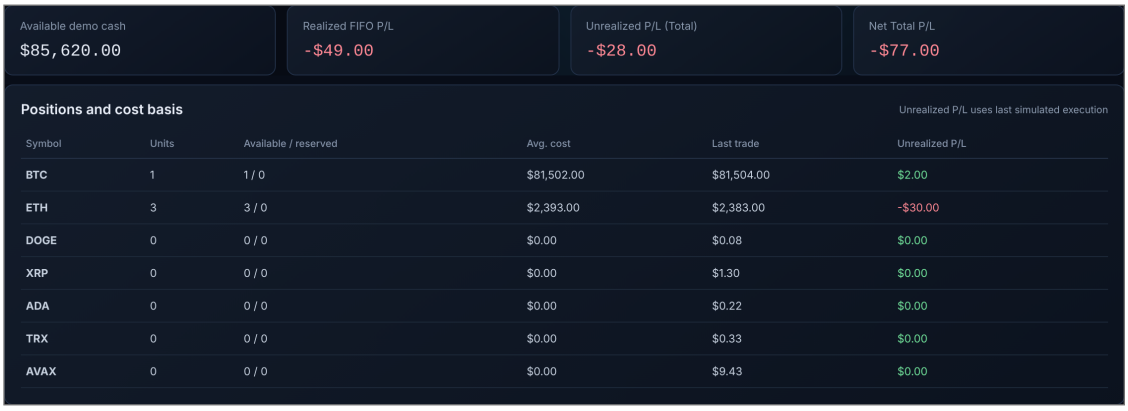


Figure 25: *Portfolio: cash, realised and unrealised profit and loss, and positions with cost basis.*

7.4 Reports

On the Reports page the user typed *btc* in the linear-search box (4 executions found), chose to sort by price with quick sort, and looked up trade id 4 by binary search, which reported '#2 too small, #4 found - BTC at $81,502.00' (Figure 26). Switching to bubble sort and sorting by quantity gives the table in Figure 27. Both sorts report the number of records and the time taken.

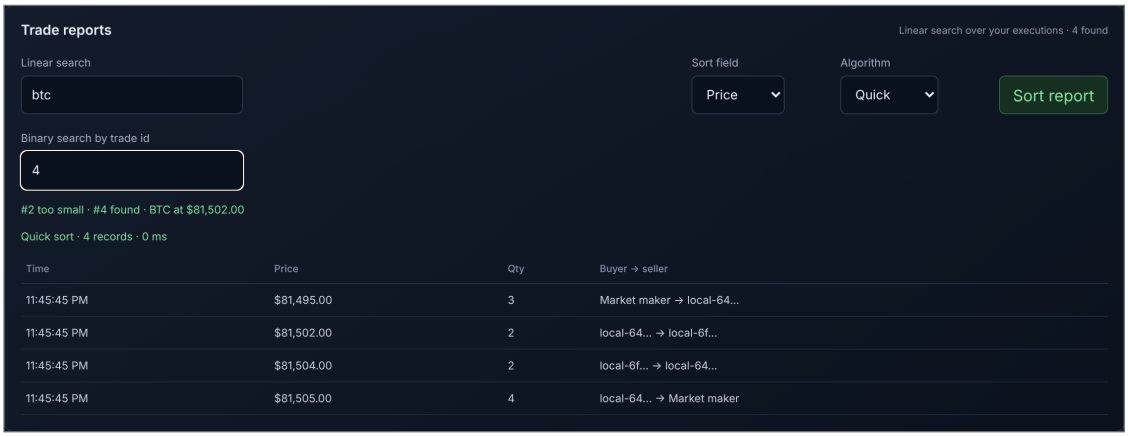


Figure 26: *Reports: linear search 'btc', binary search for trade 4, and a quick sort by price.*

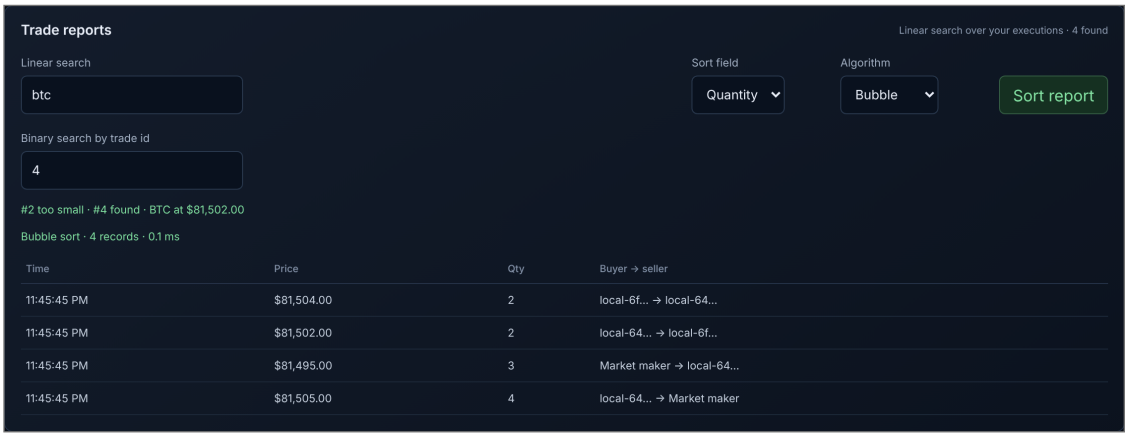


Figure 27: *Reports: the same data sorted by quantity with bubble sort.*

7.5 CSV Order Replay

The Replay page runs a CSV of up to 10,000 orders through two engines, the BST and DLL engine and a plain-array baseline, and checks that both produce the same executions. A file of 2,000 generated BTC orders was replayed (Figure 28). In this single run on the local development machine used for the screenshots, the BST and DLL engine processed 31,981 orders per second (62.54 ms) and the plain-array baseline 164,020 orders per second (12.19 ms), and the page confirmed 'Execution sequences match - 1470 executions'. **This is one observation on one machine and is not a performance claim.** The generated orders cluster within about $10 of one price, so the book has few price levels, a situation in which a simple array is competitive; the replay is intended to show that both implementations behave identically, not that either is faster in general.

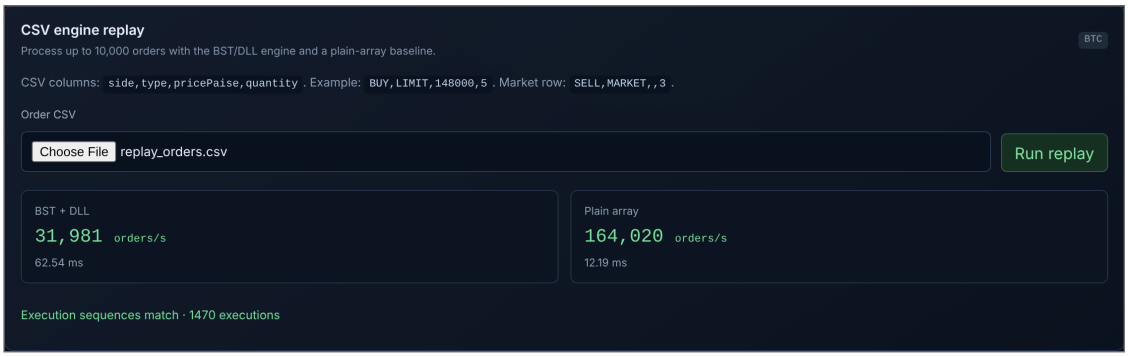


Figure 28: *CSV engine replay: timing comparison and the check that execution sequences match.*

7.6 Automated Tests

Running `npm test` in the repository (Node's test runner through `tsx --test`) executed 41 tests, all of which passed. They cover the circular list, stack and queue, the searching and sorting algorithms, order-book matching, partial fills, cancellation and several edge cases and end-to-end scenarios.

7.7 Visuals Not Included

One visual is missing from this chapter: the TradingView chart (placeholder above). The sign-in and account-creation screens of the live Neon Auth provider could not be captured either; the sign-in figure shows the application's own sign-in form from the local development build.

8. CONCLUSION

PriceTime shows that the data structures taught in a first Data Structures course are not only exercises: together they form a working trading simulator. A binary search tree keeps price levels sorted and gives the best bid and ask; a doubly linked queue at each price gives fair first-in-first-out service and constant-time cancellation; a singly linked tape records executions; a circular linked list maintains a 20-trade moving average in fixed memory; and a stack keeps an audit trail of actions. Binary and linear search, and five sorting algorithms, are applied to real order and trade data, and FIFO lot accounting turns the same first-in-first-out idea into correct realised profit and loss.

The project also taught us practical engineering: validating input, verifying signed tokens, keeping every order inside one database transaction, rebuilding state from an event log, and testing the engine with automated tests (41 tests pass). It is honest about its limits: the tree is not self-balancing, the book is rebuilt from events on each request, and PriceTime is a teaching simulator and not a real exchange. Only demo funds are used and no real cryptocurrency is bought or sold.

The next steps listed under Future Scope, such as a balanced tree, more order types and live updates, would make the system closer to production practice while keeping its educational purpose. For us, the main outcome of the project is a clear understanding of why each data structure was chosen and how its complexity affects a real application.

9. REFERENCES

Project sources

[1] PriceTime source code repository, GitHub: https://github.com/sonstellar969/PriceTime

[2] PriceTime deployed web client (GitHub Pages): https://sonstellar969.github.io/PriceTime/

Technology documentation

[3] React documentation: https://react.dev

[4] TypeScript documentation: https://www.typescriptlang.org/docs/

[5] Vite documentation: https://vite.dev

[6] Node.js documentation (including the built-in test runner, node:test): https://nodejs.org/docs/latest/api/

[7] Hono web framework documentation: https://hono.dev

[8] Zod documentation: https://zod.dev

[9] jose (JSON Web Token library) repository and documentation: https://github.com/panva/jose

[10] PostgreSQL documentation (transactions, advisory locks): https://www.postgresql.org/docs/

[11] Neon documentation (Postgres, Auth and Functions): https://neon.com/docs

[12] Drizzle ORM documentation: https://orm.drizzle.team

[13] TradingView widget documentation (Advanced Chart): https://www.tradingview.com/widget-docs/

[14] Binance Spot API documentation (public market data): https://developers.binance.com/docs/binance-spot-api-docs

[15] GitHub Pages documentation: https://docs.github.com/en/pages

[16] GitHub Actions documentation: https://docs.github.com/en/actions

Textbooks on data structures and algorithms

[17] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.

[18] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed., Addison-Wesley, 2011.

[19] M. A. Weiss, *Data Structures and Algorithm Analysis*, 4th ed., Pearson, 2013.

[20] E. Horowitz, S. Sahni and S. Anderson-Freed, *Fundamentals of Data Structures in C*, 2nd ed., Silicon Press, 2008.

[21] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed., Addison-Wesley, 1998.